

RELATÓRIO - TRABALHO 3: Web Services e API REST - Mural Pixel Art

Equipe: Jessica Clezia de Oliveira Campos, Victor Kauã Guilherme dos Santos

1. Introdução

Este relatório descreve a implementação do Trabalho 3 da disciplina de Sistemas Distribuídos. O objetivo foi implementar o serviço remoto desenvolvido no Trabalho 2 (baseado em RMI) utilizando os conceitos de Web Services e API REST, eliminando completamente o uso de sockets e RMI e adotando comunicação via protocolo HTTP com troca de mensagens no formato JSON.

O sistema é um servidor colaborativo de Pixel Art que permite a múltiplos clientes, implementados em linguagens diferentes, interagir com um mural compartilhado por meio de requisições HTTP padrão. A separação entre servidor e clientes é total: o servidor roda em um processo independente e os clientes se conectam remotamente informando o IP do host.

2. Arquitetura e Objetos Distribuídos

O servidor é implementado na classe Servidor API, que sobe um HttpServer nativo do Java na porta 8080 com um pool de dez threads. O requisito de ao menos três objetos distribuídos é atendido pelos três recursos REST registrados como contextos no servidor:

O MuralResource é responsável por criar e visualizar o mural. Ele trata requisições POST /mural (criação com largura, altura e dono) e GET /mural (retorno da representação visual, em texto puro ou colorido com códigos ANSI conforme o parâmetro ansioso). O PixelResource gerencia a pintura e remoção de pixels individuais: POST /pixel pinta um pixel na posição (x, y) com uma cor, DELETE /pixel/{x}/{y} remove o pixel da posição e GET /pixels lista todos os pixels atualmente pintados no mural. O FerramentaResource expõe as ferramentas de edição: POST /pincel aplica uma cor em uma área retangular definida por dois pontos (x1, y1) e (x2, y2), enquanto POST /borracha apaga o pixel em uma posição específica.

Todos os recursos compartilham uma instância única de PixelArt Service, que coordena as operações sobre o Mural com métodos synchronized para garantir segurança em ambiente multi-thread. O Mural também sincroniza internamente seus métodos críticos, como pintar Pixel, apagar Pixel e listar Pixels, evitando condições de corrida mesmo sob alta carga.

3. Modelo de Domínio

O domínio do sistema é representado por um conjunto coeso de classes serializáveis. A classe abstrata Usuário possui o campo nome é estendida por Artista e Administrador, que representam os diferentes papéis dos usuários do mural. A classe Coordenada encapsula as coordenadas inteiras (x, y) de um pixel. A classe Pixel associa uma Coordenada a uma cor, armazenada sempre em letras maiúsculas para normalização. A classe Mural é o objeto central do sistema: mantém a lista de pixels, as dimensões largura e altura e uma referência ao usuário dono.

A hierarquia de ferramentas segue o padrão Strategy. A classe abstrata Ferramenta define o método aplicar(Mural, x, y, cor), implementado pela Pincel (que delega para mural.pintar Pixel e oferece adicionalmente o método aplicar Área para preencher retângulos) e pela Borracha (que delega para mural.apagar Pixel). O PixelArtService instância permanentemente uma Pincel e uma Borracha, reutilizando-os em todas as requisições.

4. Protocolo de Comunicação e Serialização JSON

Toda comunicação entre clientes e servidor ocorre via HTTP com corpo em JSON. O servidor não utiliza nenhuma biblioteca externa para serialização: a classe Json Útil implementar manualmente a serialização (método escape para tratamento de caracteres especiais), a desserialização (para JsonObject que extrai pares chave-valor respeitando aspas e vírgulas dentro de strings) e os formatos padrão de resposta. Respostas de sucesso seguem o padrão {"ok":true,"mensagem":"..."} e respostas de erro seguem {"ok":false,"erro":"..."}.

A classe `Http Responder` encapsula o envio das respostas, configurando o `Content-Type` como `application/json` com `charset UTF-8` e o cabeçalho `CORES Access-Control-Allow-Origin: *` para permitir acesso de qualquer origem.

Os endpoints de leitura do mural retornam a representação visual como string JSON escapada dentro do campo mural, acompanhada dos dados estruturados do mural (dimensões, dono, lista de pixels) no campo dados. O cliente é responsável por desfazer o escape da string para exibir o mural corretamente no terminal.

5. Clientes em Múltiplas Linguagens

O trabalho exige clientes em pelo menos duas linguagens diferentes da linguagem do servidor. Foram implementados um cliente em Java e um cliente em Python, ambos funcionalmente idênticos e operando via menu interativo no terminal.

O cliente Java (`ClienteJavaAPI.java`) utiliza a API `java.net.http.HttpClient`, disponível nativamente a partir do Java 11, sem dependências externas. Ele oferece as sete operações do menu: criar mural, pintar pixel, apagar pixel, listar pixels, aplicar pincel, aplicar borracha e visualizar o mural. A visualização do mural exige tratamento especial: a string retornada pelo servidor contém sequências de escape JSON que precisam ser interpretadas para reconstruir os códigos ANSI e exibir as cores no terminal.

O cliente Python (`cliente_python_api.py`) usa exclusivamente a biblioteca padrão `urllib.request`, sem `pip install`. Oferece o mesmo menu interativo com as mesmas sete operações. Ambos os clientes aceitam o endereço IP do servidor como argumento de linha de comando (padrão `localhost`), permitindo a interação entre as máquinas da dupla.

Compilação e execução do servidor: `mvn -pl trabalho-3 package -DskipTests` seguido de `java -jar trabalho-3/target/trabalho-3-*.jar`. O cliente Java é compilado com `javac -encoding UTF-8 ClienteJavaAPI.java` e executado com `java Client Java API [host]`. O cliente Python é executado diretamente com `python cliente_python_api.py [host]`.

6. Interação entre os Estudantes

A validação do sistema foi realizada com um estudante executando o servidor e o outro executando os clientes em máquinas distintas conectadas na mesma rede local. O cliente informou o IP do servidor como argumento, e todas as operações do menu foram testadas nos dois clientes (Java e Python), confirmando o pleno funcionamento da API REST para múltiplos clientes em linguagens diferentes interagindo com o mesmo servidor Java.

7. Conclusão

O Trabalho 3 implementou com sucesso o serviço de Pixel Art usando API REST sobre HTTP. A abordagem eliminou o acoplamento rígido imposto pelo RMI, tornando o sistema interoperável com qualquer cliente capaz de realizar requisições HTTP. Os três objetos distribuídos organizaram as responsabilidades de forma coesa e a ausência de bibliotecas externas demonstra o domínio do protocolo HTTP e da serialização JSON. A concorrência foi tratada com pool de threads e métodos `synchronized`, entregando um servidor robusto e de fácil expansão.